MOSES Conversion to Object Code

Elias Lindstenz

March 2025

Introduction

The Model of the Swedish Economic System (MOSES) is one of the very first agent-based macro models (Richiardi 2024). It was designed half a century ago and has been empirically implemented on Swedish data (Eliasson 1976). An early ambition has been to give markets and firms an explicit role in explaining macro development. Its production system is therefore modelled in micro. The demand side is represented as a traditional macro expenditure system, even though the ambition has all the time been to convert also households onto a micro platform. Its model is coded in A Programming Language (APL), a high-level, array-based language with very concise syntax. Although the original code has functioned well over the years, APL is not the best programming language to build a platform for integrating production firms and households at the micro market level. There are thus strong reasons for converting the model to a modern object-code in preparation for a micro household extension. Object code offers an intuitive structure that is more easily understood and that increases code readability and extensibility.

The first conversion to object code, based on early pseudocode in *A Micro-Macro Interactive Simulation Model of The Swedish Economy* (Eliasson, 1976) was completed in 2024. A second conversion, based on the APL project and pseudocode in "The MOSES Technical Specification Code" (Fredrik Bergholm et al., 1989) is now complete.

The C++ language was chosen for its speed, object structure, popularity, and the author's familiarity with the language.

An in-depth technical specification and pseudocode of the model is in *MOSES C++ Technical Specification* (Lindstenz, 2025). This is a new C++-adapted pseudocode that is different from the revised version of the APL-adapted pseudocode from 1989 (Lindstenz, 2024).

Object code

Array-based languages like APL use arrays for representation. In chaotic simulation models such as MOSES, arrays are a simple solution. Values are stored in arrays that are globally accessible for the most part, and each value typically represents an object. For example, one array keeps track of the labour force of all firms. Designing a consistent structure in object code is more difficult since chaotic systems are heavily intertwined. For example, taxes,

associated with the government, are calculated in firm and household specific equations. Furthermore, some values are not associated with specific objects. Aggregate interest receipts from firms and total quarterly export in the economy do not refer to specific objects for instance.

The benefits of an object structure outweigh the initial trouble of designing a consistent structure. Object-oriented programming languages like C++ use classes, which are blueprints for any number of instances of a type of object. For example, a firm class can instantiate an infinite number of firms, all with their own specific values for revenue, labour, wages, etc. This is a more intuitive way to represent behaving economic actors in a micro macro simulation. Each firm or household is one object, instead of an aggregate of values from a great number of arrays.

In addition to the cognitive ambition to facilitate visualisation and understanding, the inherently intuitive structure of object code facilitates the development of MOSES by increasing readability and code extensibility. Readability refers to the developer's ability to read and understand code. APL code is not very popular today. It has an incredibly concise syntax that is not easily understood for coders of other languages. The right-to-left execution of code, and especially the use of unusual characters reduces the readability of APL.

Extensibility is the level of ease for code to be extended or updated. By limiting access between actors in the economy, classes localise object-specific behaviour and prevent accidental interactions. For example, the firm's production plan is handled by the firm alone, with limited external dependencies. This structure can accurately simulate reality. Since behaviour is localised modularly, code also becomes more easily extensible. Functionalities can easily be added independently of existing functionalities. In contrast to array-based languages like APL, actors in a C++ coded model economy are represented as independent agents that contain functionality within themselves, unless otherwise specified.

The structure of object code facilitates the integration in MOSES of a micro household sector since it functions modularly, and with localised behaviour, which is exactly what object code is built for. Lastly, the popularity of C++ is positive if the project should be further developed by others.

The object structure in the most updated C++ model was inspired by Derviz (1992) who investigated the potential of converting MOSES to object code, which at that time was found to be a much larger project than it is today.

A Micro Household Sector

Object code facilitates the representation of households in micro simulations by using classes to represent households as independent agents. Micro specified households may be converted to business entities where household members contribute human competence and financial resources that may later hire workers, borrow money and issue equity to expand into growing business firms. Something like this was attempted in Ballot, Eliasson & Taymaz (2006) who

made venture capital firms spin off as financial businesses from MOSES' macro household sector. If such a household business fails, it regresses to the household sector. There are infinite possibilities to model firm birth, life and death like this. Adding such diversified characteristics of business formation will make a more theoretically and empirically interesting and analytically capable model of MOSES. For example, it will be possible to study the relationships between economic growth and income and wealth distributions.

Model Observations

Being a highly integrated market selection based non-linear economic system, the current C++ MOSES model is heavily dependent on high quality input data on its initial state for proper simulation. Inconsistent data and measurement errors cause exponentially unrealistic effects. A micro household sector that allows the creation of new firms will likely stabilise model simulations since a continuous volume of new firms will stabilise labour unemployment. For example, as firms fail and optimise labour utilisation, unemployment increases indefinitely. A flow of new firms will employ from the unemployment pool. Labour unemployment is a key variable in this chaotic system, affecting consumer spending, which may have butterfly effects. Disallowing extreme unemployment will therefore likely have strong stabilising effects on the model.

C++

The C++ programming language was chosen for its speed, object structure, popularity, and the author's familiarity with the language. C++ is an incredibly efficient language that balances low-level speed with abstractness. While making an object structure that allows for abstract modelling, it also gives the same low-level access for efficiency as its predecessor C. Additionally, the syntax of the programming language is easily understood as is the case with many other popular languages today. Code is read in the same way as its logic is executed. These are some of the reasons why C++ is so popular despite being almost 50 years old. The positives of C++ far outweigh the increased conciseness of APL.

References

Ballot, Gerard, Gunnar Eliasson and Erol Taymaz, 2006, *The Role of Commercialization Competence in Endogenous Economic Growth,* revised version of paper presented to the International J.A. Schumpeter Society 11th ISS Conference, Nice-Sophia Antipolis, 21-24 June 2006, to be republished in the Int. Journal of Microsimulation, 2024.

Derviz, Alexis, 1992, “Investment and Knowledge Transfer in a MOSES Economy – An approach Based on Net Theory and Object Oriented Programming “, (Institute of Information Theory and Automation, Prague and IUI Working Paper)

Eliasson, Gunnar, 1976, "A Micro-to-Macro Model of the Swedish Economy, Preliminary Documentation," Economic Research Report B15. Stockholm: Federation of Swedish Industries. Reprinted in *International Journal of Microsimulation*, 2024, 17(2).

Lindstenz, Elias, 2025, *MOSES C++ Technical Specification*.

Lindstenz, Elias, 2024, *The MOSES Technical Specification Code – Integrating the 1976 and 1989 versions*. Available as supplement to “Modelling the Experimentally Organized Economy (II)” in *International Journal of Microsimulation*, 2024, 17(2); 1-59. https://github.com/microsimulation/ijm-supplements/tree/ijm-00291.

Richiardi, Matteo, 2024, “Editorial”, *International Journal of Microsimulation*, 2024, 17(2); I-III

Appendix

The C++ MOSES project can be accessed from the author’s GitHub repository: https://github.com/novrion/MOSES.